INSPIRE: Accelerating Deep Neural Networks via Hardware-friendly Index-Pair Encoding

Fangxin Liu[1,2,†], Ning Yang[1,2,†], Zhiyan Song[2], Zongwu Wang[1,2], Haomin Li[1,2], Shiyuan Huang[1,2], Zhuoran Song[1], Songwen Pei[3] and Li Jiang[1,2]
1. Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China
2. Shanghai Qi Zhi Institute 3. University of Shanghai for Science and Technology

ABSTRACT

Deep Neural Network (DNN) inference consumes significant computing resources and development efforts due to the growing model size. Quantization is a promising technique to reduce the computation and memory cost of DNNs. Most existing quantization methods rely on fixed-point integers or floating-point types, which require more bits to maintain model accuracy. In contrast, variable-length quantization, which combines high precision for values with significant magnitudes (i.e., outliers) and low precision for normal values, offers algorithmic advantages but introduces significant hardware overhead due to variable-length encoding and decoding. Also, existing quantization methods are less effective for both (dynamic) activations and (static) weights due to the presence of outliers.

In this work, we propose INSPIRE, an algorithm/architecture co-designed solution that employs an Index-Pair (INP) quantization and handles outliers globally with low hardware overheads and high performance gains. The key insight of INSPIRE lies in identifying typical features associated with important values, encoding them as indexes, and precomputing corresponding results for efficient storage in lookup table. During inference, the results of inputs with paired index can be directly retrieved from the table, which eliminates the need for any computational overhead. Furthermore, we design a unified processing element architecture for INSPIRE and highlight its seamless integration with existing DNN accelerators. As a result, INSPIRE-based accelerator surpasses the state-of-the-art quantization accelerators with a remarkable 9.31× speedup and 81.3% energy reduction, respectively, while maintaining superior model accuracy.

ACM Reference Format:
Fangxin Liu[1,2,†], Ning Yang[1,2,†], Zhiyan Song[2], Zongwu Wang[1,2], Haomin Li[1,2], Shiyuan Huang[1,2], Zhuoran Song[1], Songwen Pei[3] and Li Jiang[1,2]. 2024. INSPIRE: Accelerating Deep Neural Networks via Hardware-friendly Index-Pair Encoding. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3655896

† These authors contributed equally.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3655896

1 INTRODUCTION

Deep Neural Networks (DNNs) have achieved remarkable success in fields like computer vision and natural language processing [5, 23]. However, such performance boost has been achieved at the cost of extremely energy-intensive DNN models due to the increasingly larger model size [9]. For instance, state-of-the-art DNNs like GPT-3 may require up to GBs (Giga Bytes) for model size and 10^2 GFLOPs (Giga Floating Point Operations) for inference computation [2].

Therefore, deployment of such DNNs remains a challenge, requiring two crucial supports. The first one is specialized hardware accelerations for DNN inference, such as Eyeriss [3], BitFusion [19] and TPU [11]. The second is model compression techniques, which explore the opportunity of algorithm and hardware co-design to achieve better trade-offs between accuracy and hardware overhead. Model quantization is one of the most hardware-efficient compression techniques to reduce inference costs for large DNN models [6]. It represents model parameters with fewer bits to simplify the implementations and accelerate the inference execution speed in a hardware-friendly manner. Also, it is supported in GPU tensor core [15] and mainstream accelerator architectures like systolic array [3, 11].

However, existing quantization schemes are less effective when applied to DNNs that consider both activations and weights, especially for larger models. This is because the values in DNN tensors have a non-uniform distribution and non-uniform importance [13, 24]. Model performance is particularly sensitive to only a very small fraction of outliers, particularly in dynamic activations, which are far more significant than normal values [8, 12, 14]. Simply clipping outliers could result in a significant reduction in model accuracy, while including these outliers directly in the quantization range could lead to a cumulative increase in the rounding error. Consequently, a common practice is to use larger bitwidths (e.g., 8-bit or more) to quantize DNN models, such as activations.

In an effort to construct faster DNN models, many researchers have explored outlier-aware algorithm-and-accelerator co-designs that focus on quantization [8, 21, 24]. These designs incorporate hardware designs that can efficiently represent normal and outlier values with different precision. However, they are not compatible with existing DNN accelerators due to their variable-length encoding and unaligned memory access, requiring costly and complex hardware designs to support.

Previous designs often prioritized the reduction of bitwidths to simplify quantization, but this led to difficulties in accurately representing activation values [12]. Outliers would stretch the quantization range, causing most values to be compressed into just a few effective bits [24]. However, our paper presents a novel approach utilizing centroids to represent parameter distributions, rather than

quantization intervals [9]. Through this method, we create a global quantization process that can adapt to the varying importance of different values.

To optimize the performance and efficiency of deep neural networks, we propose a pioneering approach that leverages centroids and their associated indexes for aligned memory access and low-bit computation. By using indexes to record centroids in the codebook, we can maintain efficient quantization while only requiring fixed-size, low-precision int data types in the network. Furthermore, we present a novel index-pair processing element (IP-PE) design that can accommodate weights and activations based on indexes, not values, which is well-suited for lookup tables (LUTs). By simplifying PEs to support index-pair matching and utilizing a unified formatting scheme to compute on fixed-size, low-precision indexes, our approach delivers remarkable reductions in computation cost for inference. We make the following contributions in this paper:

- We propose INSPIRE as a novel approach for efficient encoding of activations and weights, offering a distinctive inference paradigm based on index representation data type.
- We propose the efficient architectural implementation and integration of INSPIRE quantization, specifically designed to handle cases where operators involve indexes rather than values.
- We evaluate the effectiveness of INSPIRE by comparing it with existing quantization accelerators. The results demonstrate the superiority of INSPIRE, achieving up to a 81.3% reduction in energy consumption and a 9.31× speedup while maintaining the similar accuracy as full-precision models.

2 BACKGROUND AND MOTIVATION

2.1 Outlier Matters in Quantization

We visually illustrate the significance of DNN outliers in Figure 1, using ResNet-50 as a representative CNN model and BERT for the attention-based model. The outliers in DNNs are ≈ 100× larger than most activation values [12, 24]. During quantization, these large outliers dominate the maximum magnitude measurement, resulting in low effective quantization levels for normal values and, consequently, substantial quantization errors. A notable observation is the significant difference in outlier magnitudes between attention-based and CNN models. Attention-based models exhibit outliers that are an order of magnitude larger than those in CNNs. While retraining algorithms can restore accuracy in CNN models even when outliers are clipped under ultra-low-bit precision (e.g., 4-bit), this proves to be more challenging for attention-based models due to their significantly larger outliers [8, 18].

2.2 Model Quantization

The research focus on outliers has spurred the development of outlier-aware quantization techniques [8, 24]. These methods typically utilize low-precision 4-bit integers for small, frequently occurring values and high-precision 32/16-bit floats or integers for infrequent, large values. For example, GOBO [25] and OLAccel [21] utilize a coordinate list to pinpoint outlier locations, allowing representation with high precision while compressing normal values with fewer bits. In BiScaled-DNN [10], all values share the same bit-width but differ in scale factors for normal values and outliers.

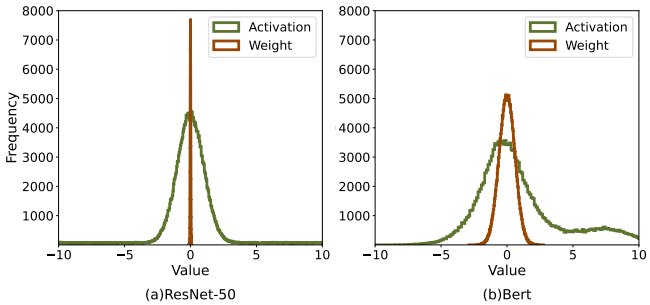


Figure 1: Distribution of activation/weight values in a single layer for the (a) ResNet-50 and (b) BERT model.

DRQ [20] employs a bitmap and uses mixed precision (4-bit and 8-bit) for data storage. ANT [8] uses mixed data types to better fit distributions, yet introducing complex logic for decoding the data type. However, these quantization techniques exploit variable-length data encoding, which leads to the non-alignment in the memory sub-system or complex hardware logic.

2.3 Motivation

Weight distribution is generally uniform and flat, making it easy to quantize. Previous studies have demonstrated that quantizing LLM weights with 8-bit or even 4-bit precision does not compromise accuracy, aligning with our observation [8, 12, 14, 24]. However, the presence of outliers in DNNs poses challenges for activation quantization. Consequently, the conventional approaches involve adopting larger bitwidths, such as 8-bit or 16-bit, for activation quantization.

To this end, we explore an alternative approach inspired by the observation that clustering DNN values and learning centroids adaptively could be advantageous. However, this method introduces the need for additional indexes to record centroid locations. In response, our paper introduces a new inference paradigm based on these indexes, departing from the conventional quantization architecture that relies on bit-width. This paradigm replaces traditional computation operators with table lookup. With the preset quantization patterns, we precompute centroid results and store them offline in tables. During inference, matched centroids with inputs can be efficiently retrieved from the table, offering a more adaptive and efficient approach.

3 INSPIRE ALGORITHM

In this section, we introduce INSPIRE, our adaptive encoding approach that efficiently leverages intra-layer adaptive opportunities to globally fit the distributions of activations and weights. We propose a novel encoding scheme leveraging centroids to accommodate varying value significance within a layer. Additionally, we propose a dynamic framework that adapts to each layer's distribution by selecting appropriate centroids. Consequently, INSPIRE ensures aligned memory accesses and efficient low-bit computation, delivering significant benefits with minimal hardware overhead.

3.1 Algorithm Overview

Our approach to compressing model parameters deviates from conventional methods like sparsity or quantization. Rather than aiming for compression to specific bitwidths or maintaining a high sparsity

rate, we employ clustering to identify centroids that adapt to the importance of different parameter values. This strategy enables efficient processing of both outlier and normal values. Our objective is to identify the key factors (i.e., centroids) in the parameter distributions of DNN to minimize model loss.

For a given activation or weight matrix, the k-means clustering method, based on a similarity measure, proves to be the most effective for compression. We select initial C clustering centers and iteratively update them by classifying data into different clusters until minimal error variation is achieved. However, the direct application of the k-means clustering method faces challenges. The clustering effect depends on initialized centers and is sensitive to outliers. Moreover, while a smaller C can enhance compression, k-means, being an unsupervised learning method, cannot guarantee compressed accuracy.

To overcome these challenges, we introduce optimizations. First, we uniformly choose the initial C values over a range. Since uniform quantization is already effective, starting clustering from uniformly quantized values yields better results. Furthermore, we employ the Gap statistic method [22] to determine the initial C value, ensuring a balanced trade-off between compression quality and efficiency:

$$Gap(C) = E(\log D_C) - \log D_C \tag{1}$$

where $E(\log D_C)$ is obtained through the following process: we uniformly generate a number of random data points equal to the original dataset size within a defined range. k-means clustering is then performed on this random sample to obtain D_C. This process is repeated multiple times to obtain several $\log D_C$ values, and their average approximates $E(\log D_C)$. The initial number of centroids, determined by maximizing Gap(C), is obtained through this process.

Subsequently, based on the initial conditions, we employ k-means clustering to process weights from the pre-trained model and activations from random training samples to determine centroids. We then conduct inference to test the compressed model and obtain accuracy. Iteratively adjusting C based on observed accuracy losses, we identify the optimal activation C_A and weights C_W clusters.

3.2 Encoding and Indexing

After determining the centroids offline, we proceed with the compression and computation using these centroids. The centroid computation is conducted offline, utilizing pre-trained weight parameters and predetermined activations obtained from the training set. For a value x, we determine its compression result by finding the centroid with the smallest L1 norm absolute difference:

$$Compressed(x) = \min_i |C_i - x| \tag{2}$$

For static weight parameters, we precompute the calculations offline to optimize efficiency. On the other hand, for dynamic input activations, we perform online calculations to determine their compressed values, leveraging the pre-determined centroids and the L1-norm metric. To further enhance performance, we order the centroids from smallest to largest and initiate a binary search from the middle item. This binary search strategy effectively reduces the computation complexity from C to $\log C$. Moreover, the activation encoding process can be implemented in a pipeline, ensuring no increase in time cost.

After determining the centroids, we observe an opportunity to simplify the computation of the activation-weight matrix. Given that centroid values are predetermined and limited, we can precompute the multiplication results of compressed activations and weights, storing them in a lookup table. This eliminates the need for multiplication during actual operations, as the results can be directly read. Taking advantage of this optimization, we convert the encoding results from centroid values to their corresponding indexes. Through the construction of index-pairs for activation and weight indexes, a lookup table is formed, facilitating direct index usage for data transmission and result retrieval. INSPIRE, our compression scheme, follows a two-phase process outlined in Algorithm 1: 1) In phase 1, we utilize the pre-trained weight parameters and a subset of the training data to calculate and determine the number of clusters C_a and C_w along with the corresponding clustering centroids. We encode indexes in the order of their numerical values $[0, 1, \ldots, C-1]$. 2) In phase 2, we convert the parameters directly to the corresponding indexes through binary search. We perform searches directly through the indexes to obtain the calculation results and carry out accumulation and output.

4 INSPIRE ARCHITECTURE

In this section, we describe the incorporation of INSPIRE into the systolic array architecture. The unique encoding introduced by INSPIRE presents challenges in designing the processing element (PE), as it now works with indexes rather than values for computations. To this end, we introduce the IP-PE architecture, which replaces the multiply-accumulate (MAC) operation among values featuring data types with a table lookup based on indexes. Additionally, we present the hardware encoder tailored for the aforementioned index-pair encoding.

4.1 Architecture Overview

Figure 2 provides an overview of the INSPIRE architecture, consisting of multiple PE pages. Each PE page holds two buffers for storing the indexes of encoded activations and weights. The weights buffer contains preprocessed weight data entered into the PE array in advance. INSPIRE adopts the IP-PE design, based on index-pair matching, instead of the conventional PE design. The results are obtained through search, pass through the accumulation unit, and enter the encoder. The encoder transforms the results into indexes, which are then outputted through the output buffer.

The INSPIRE architecture includes a controller, an accumulation unit, and associated peripheral circuitry. As INSPIRE encodes based on indexes, there is no need for a decoder in the architecture, resulting in a significant reduction in computational load. The detailed description of IP-PE and the encoder highlights how INSPIRE optimally leverages the benefits of neural network compression.

4.2 IP-PE Design

As previously discussed, with the offline encoding of activations and weights completed, matrix computations now leverage a LUT instead of the traditional PE. The search operation, driven by the weight-activation index pair, efficiently retrieves the corresponding computation results, as depicted in Figure 2(c). Furthermore, given that the computation involves a fixed weight pattern (one

Algorithm 1 INSPIRE Encoding Algorithm

Input: Pre-trained model M, Training dataset D, Online input activation X, Accuracy loss constraint $\mathcal{L}$ (by user).
Output: Indexed model M_q and a codebook T.

{Phase-1: Get the centroids.}
$C_A, C_W \leftarrow \text{GapStatistic}(M, D)$
▷ Get initial number of centroids.
while $(\mathcal{L}_q \geq \mathcal{L})$ **do**
  for layer $l := 1$ to L **do**
    $V_A^l, V_W^l \leftarrow \text{Clustering}(C_A, C_W, M, D)$
  end for
  $Q_W \leftarrow \text{Compress}(C_W, M)$
  ▷ Get the compressed model.
  $\mathcal{L}_q \leftarrow \text{Evaluate}(Q_W, V_A)$
  $C_A, C_W \leftarrow \text{Update}(\mathcal{L}_q, \mathcal{L})$
  ▷ Increase or decrease C depending on the accuracy loss.
end while
$I_A, I_W \leftarrow \text{Indexing}(C_A, C_W, V_A, V_W)$
▷ After find the optimal centroids, do indexing on activation and weight.
$M_q \leftarrow \text{Indexing}(I_W, Q_W)$
▷ Convert the compressed model to indexed model.
$T \leftarrow \text{Compute}(I_A, I_W, V_A, V_W)$
▷ Make the index pair codebook.
{Phase-2: Fast online encoding and calculation}
while Have input activation $x \in X$ **do**
  $i_x \leftarrow \text{BinaryIndex}(x, V_A)$
  ▷ Convert to index by binary search among activation centroids.
  $r_x \leftarrow \text{Lookup}(i_x, i_w, T)$
  ▷ Get calculation results by index pair matching.
end while
$R \leftarrow \text{Accumulate}(r)$
▷ Accumulate and output.
return Indexed model M_q and a look-up table T.

column at a time), the LUT design of the IP-PE is further optimized. Sequentially fixing the weights in the LUT ensures that only results corresponding to different activations with this weight are retained. This optimization significantly reduces the number of LUT entries recorded in a single IP-PE from $C_W \times C_A$ to C_A, resulting in a noteworthy improvement in search speed when searching multiple IP-PEs simultaneously. This approach also markedly reduces the area occupied by a single IP-PE, similar to the bucket method based on the weight centroids, as illustrated in Figure 2(b). Even if the platform lacks support for such a flexible configuration, the IP-PE can maintain $C_W \times C_A$ table entries, given the small number of clusters and to avoid becoming a bottleneck in the DNN computation.

4.3 Encoder Design

We design the encoder, as depicted in Figure 3. A binary discriminator tree with $\log C$ layers is employed to encode C centroids, where each node is equipped with a comparator. In such a tree, leaf nodes save the centroid values and non-leaf nodes save average number of their child nodes. The comparator compares the input

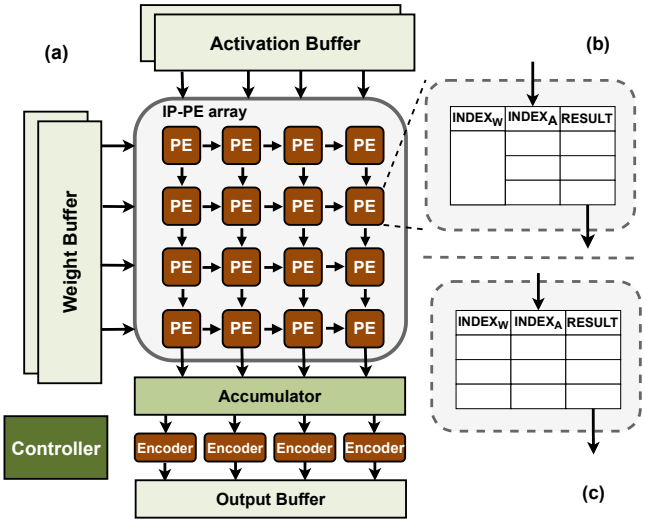


Figure 2: (a) Overview of INSPIRE architecture. (b) The IP-PE design of with LUT entries $1 \times C_A$. (c) The IP-PE design of with LUT entries $C_W \times C_A$

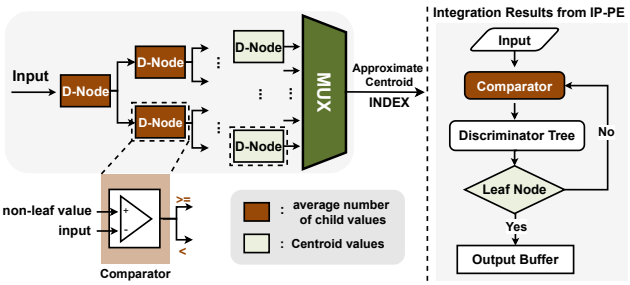


Figure 3: Left: The design of INSPIRE encoder. Right: The computation process of encoding.

with the current node value, the result of which decides which branch input data will flow into. This process will not finish until the input reaches the leaf node storing the centroid whose index will be recorded as the encoding result. The overall computation process iterates $\log C$ times through the nodes. In the pipelined computation process, the comparison steps do not become the bottleneck. Only one node of a layer is activated each cycle, allowing the rest of the nodes to remain inactive in the computation, thereby maximizing energy consumption savings.

4.4 Index-based Computation Support

The INSPIRE encoding significantly reduces the number of parameters that requires storage through the utilization of clustering methods, thereby minimizing overhead. To further mitigate transmission and storage costs, we convert compressed values to indexes, replacing high-precision values with low-precision indexes in the computation. Additionally, instead of resource-intensive multiplications, we opt for a fast index-pair table lookup to reduce computational and latency overhead.

Simultaneously, our proposed INSPIRE-based encoding supports the systolic array architecture by introducing the innovative IP-PE with a LUT and optimized encoder for efficient result searching.

Table 1: Accuracy results of evaluated model and dataset.

Type	CNN-based			Attetion-based	
Model	VGG16	ResNet18	ResNet50	ViT	BERT
Dataset	ImageNet				SST-2
FP-32 Acc.(%)	71.59	69.76	76.15	84.19	90.45
INSPIRE Acc.(%)	71.03	68.91	75.55	83.33	89.95
$\#C_A$	12	16	16	54	64
$\#C_W$	6	5	9	32	54

This design enhances pipelining and parallelization, maximizing acceleration through co-design of algorithms and architectures.

5 EXPERIMENTS

5.1 Experimental Methodology

Benchmark. To validate that INSPIRE can effectively accelerate DNNs, we evaluate both CNN and attention-based models, including computer vision and natural language processing tasks. For computer vision, we use VGG-16, ResNet-18, ResNet-50, and the attetion-based model ViT[7] on the ImageNet dataset [5]. For natural language processing, we evaluate BERT-Base with eight datasets from the GLUE dataset suite [23]. We exploit pre-trained network models from TorchVision and Huggingface Model Zoo, and their validation accuracy with FP32 as baseline.

Baselines. We implement the INSPIRE encoding framework in PyTorch [17]. We evaluate five baselines compared against INSPIRE, including: 1) Eyeriss [3], a spatial energy-efficient dataflow architecture employing coarse-grained INT16 quantization throughout the network, serving as the baseline for standard NN accuracy; 2) BitFusion [19], an accelerator with a composable MAC unit that can change quantization at the layer granularity, maintaining accuracy with coarse-grained INT16 quantization throughout the network; 3) OLAccel [16], a state-of-the-art accelerator based on outlier-aware low-precision computation; 4) AdaFloat [21], requiring an 8-bit floating-point data type to maintain the original model accuracy; 5) ANT [8], a hardware-friendly quantization accelerator combining power-of-two data types and INT types for low bit-width. For models that are not available in their paper, we reproduce their experiments and report the results.

Accelerator Implementation. We implement the INSPIRE IP-PE and encoder described in Section 4 with the Verilog RTL. Meanwhile, we use the 28 *nm* TSMC technology library and Synopsys Design Compiler [4] to study the area and energy of those components that we designed. In addition, for a fair comparison, we use the same global buffer capacity (2 MB) and memory bandwidth for all these accelerators and use CACTI [1] to estimate it that can satisfy our design goals. We use DeepScaleTool to scale all designs to the 28 *nm* process for the iso-area comparison. To evaluate the performance of our proposed INSPIRE architecture, we develop a cycle-accurate simulator to simulate the PE array and output accumulation together with the encoder. The INSPIRE architecture can be scaled to a larger number of PEs under the same area budget.

5.2 Experimental Results

Accuracy Results. We apply the INSPIRE coding algorithm, as described in Section 3, to compress various pre-trained models. The results in Table 1 reveal that activations and weights can be effectively clustered to just a dozen or even a few centroids, equivalent to achieving compression to 3-4 bits. Notably, this compression retains a similar level of accuracy to the original models. Specifically, INSPIRE employs 12 centroids for activation and 6 centroids for weight in VGG16, demonstrating impressive compression performance. Similarly, for attention-based models, INSPIRE effectively limits clustering to a relatively low level.

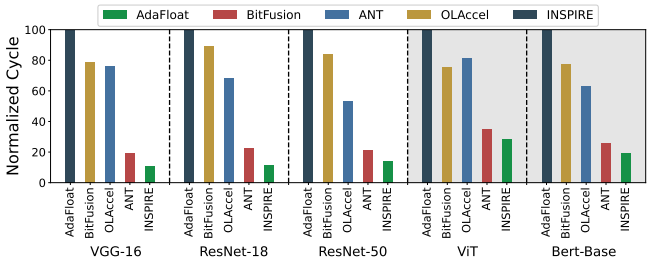


Figure 4: Performance comparison for different networks.

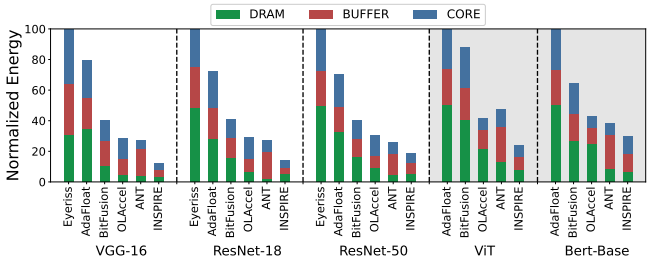


Figure 5: Energy comparison for different networks.

On one hand, the INSPIRE encoding substantially reduces the storage demand for neural network weights, transitioning from FP-based values to INT-based indexes ($\leq$ 5-bit). On the other hand, although compressing activations poses a more intricate challenge, INSPIRE adeptly lessens the number of activated centroids to represent activations. These findings highlight the efficacy and scalability of the proposed INSPIRE.

Performance. Figure 4 illustrates the normalized execution cycles of INSPIRE and other methods on different networks. From the plot, INSPIRE exhibits a significant advantage in terms of performance improvement. In comparison, INSPIRE achieves up to 9.31×, 7.95×, 7.06×, and 1.97× speedup values over AdaFloat, BitFusion, OLAccel, and ANT. Meanwhile, the performance improvement of INSPIRE is more pronounced on DNNs with more concentrated clustering and demonstrates excellent performance on attention-based models. Specifically, INSPIRE can achieve up to an 89.2% performance improvement on VGG-16. This substantial improvement is attributed to the index pair encoding algorithm in INSPIRE, allowing the replacement of the 4-bit or 8-bit PE multiplication with IP-PE, an efficient lookup table design. The IP-PE further saves the area of a single PE, enabling the deployment of more IP-PEs within the same area budget, resulting in better speedup.

Energy Figure 5 compares the normalized energy consumption of INSPIRE and other works. The energy consumption is decomposed into DRAM, global buffer, and processing core (Core). For ResNet-50, INSPIRE consumes up to 81.3% less energy than Eyeriss and up to 27.7% less than ANT. For Bert, INSPIRE consumes

Table 2: The configuration and area breakdown of INSPIRE and other works under 28 nm process.

Architecture	Core		
	Component	Number	Area(mm^2)
INSPIRE	Encoder(12.44 μm^2)	128	0.165
	IP-PE(9.95μm^2)	16,384	
ANT	Decoder(4.9μm^2)	128	0.327
	4-bit PE(79.57 μm^2)	4,096	
BitFusion	4-bit PE	4,096	0.326
OLAccel	4-bit & 8-bit PE	1,152	0.309
AdaFloat	8-bit PE	896	0.327
Eyeriss	16-bit PE	168	0.309

69.8%, 53.4%, 30.1%, and 21.8% less energy than AdaFloat, BitFusion, OLAccel, and ANT, respectively.

This energy advantage primarily comes from optimizing IP-PE by substituting the costly multiplication operation with encoder subtraction and table lookup, thereby significantly reducing energy consumption. Additionally, INSPIRE stores only indexes in the buffer, offering it a substantial advantage over other schemes.

Area. Table 2 shows the area breakdown of INSPIRE under 28 nm process, with other accelerators scaled to the same process for comparison. The results indicate that the encoder just occupy a small fraction of the area, about 0.96% of the overall device. And the encoding strategy enables low hardware overhead through the implementation of Table Lookup in IP-PE. Overall, with a similar area budget, INSPIRE accommodates a larger number of IP-PE. The small area overhead of our INSPIRE directly benefits from the carefully designed index pair encoding and computation based on the table lookup.

Impact of Centroid Number. The number of centroids impacts not only the inference efficiency but also the model accuracy. Figure 6 presents accuracy and normalized energy for different centroid numbers in ResNet-50. Generally, a smaller number of centroids results in fewer entries in IP-PE, reducing searching costs but potentially degrading model accuracy. Additionally, it is observed that activation is more sensitive to the number of centroids compared to weight. Therefore, a higher number of centroids is required for activation to maintain accuracy. The energy consumption primarily depends on the bitwidth required for the centroid index, impacting both storage and encoding overhead. The trade-off involves choosing an optimal solution that balances accuracy and energy consumption.

6 CONCLUSION

This paper introduces INSPIRE, an algorithm/architecture co-design framework poised to facilitate efficient DNN inference. Central to this approach is the strategic utilization of the table lookup, simplifying the inference software and hardware design and decoupling with the table updates. By the centroid quantization technique for DNN, INSPIRE achieves comparable accuracy for complex tasks with much less resource cost. Experiments demonstrate that INSPIRE can yield satisfactory performance gains with trivial accuracy loss, making it a promising solution for the DNN deployment.

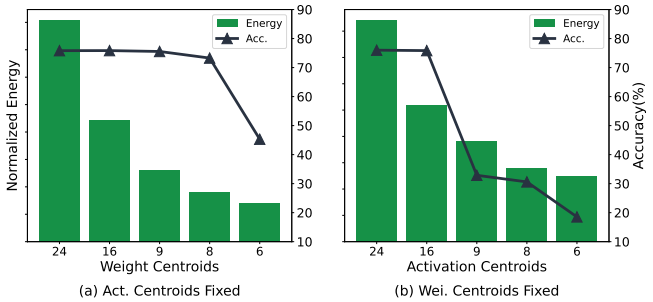


Figure 6: Accuracy and normalized energy with different number of centroids. The Centroids of both activation (a) and weight (b) are fixed to 16.

ACKNOWLEDGMENTS

We thank Huan Zhou for improving the figures. This work was partially supported by the National Natural Science Foundation of China (Grant No. 61975124, 62202288). Li Jiang is the corresponding author (ljiang_cs@sjtu.edu.cn).

REFERENCES

[1] Rajeev Balasubramonian et al. 2017. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *TACO* (2017).
[2] Tom Brown et al. 2020. Language models are few-shot learners. *NIPS* (2020).
[3] Yu-Hsin Chen et al. 2019. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *JETCAS* (2019).
[4] Synopsys Design Compiler. 2019. [Online]. Available: https://www.synopsys.com/support/training/rtlsynthesis/design-compiler-rtl-synthesis.html.
[5] Jia Deng et al. 2009. Imagenet: A large-scale hierarchical image database. In *CVPR*.
[6] Lei Deng et al. 2020. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proc. IEEE* (2020).
[7] Alexey Dosovitskiy et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
[8] Cong Guo et al. 2022. Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *MICRO*.
[9] Benoit Jacob et al. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *CVPR*.
[10] Shubham Jain et al. 2019. BiScaled-DNN: Quantizing long-tailed datastructures with two scale factors for deep neural networks. In *DAC*.
[11] Norman P Jouppi et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *ISCA*.
[12] Sangil Jung et al. 2019. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *CVPR*.
[13] Fangxin Liu et al. 2021. Improving neural network efficiency via post-training quantization with adaptive floating-point. In *ICCV*.
[14] Fangxin Liu et al. 2024. SPARK: Scalable and Precision-Aware Acceleration of Neural Networks via Efficient Encoding. In *HPCA*.
[15] Nvidia. 2020. Ampere Architecture Whitepaper. https://images.nvidia.cn/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf.
[16] Eunhyeok Park et al. 2018. Energy-efficient neural network accelerator based on outlier-aware low-precision computation. In *ISCA*.
[17] Adam Paszke et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *NIPS* (2019).
[18] Jennifer Scott and Miroslav Tuuma. 2023. *Algorithms for sparse linear systems*. Springer Nature.
[19] Hardik Sharma et al. 2018. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network. In *ISCA*.
[20] Zhuoran Song et al. 2020. Drq: dynamic region-based quantization for deep neural network acceleration. In *ISCA*.
[21] Thierry Tambe et al. 2020. Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference. In *DAC*.
[22] Robert Tibshirani, Guenther Walther, and Trevor Hastie. 2001. Estimating the number of clusters in a data set via the gap statistic. *J R STAT SOC B* (2001).
[23] Alex Wang et al. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv* (2018).
[24] Guangxuan Xiao et al. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *ICML*.
[25] Ali Hadi Zadeh et al. 2020. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *MICRO*.

Ink: Efficient Incremental k-Critical Path Generation

Che Chang
University of Wisconsin at Madison
Madison, Wisconsin, USA

Tsung-Wei Huang
University of Wisconsin at Madison
Madison, Wisconsin, USA

Dian-Lun Lin
University of Wisconsin at Madison
Madison, Wisconsin, USA

Guannan Guo
University of Illinois
Urbana-Champaign, Illinois, USA

Shiju Lin
The Chinese University of Hong Kong
Hong Kong, China

ABSTRACT

Critical Path Generation (CPG) is crucial for static timing analysis (STA) applications to validate timing constraints. Recent years have witnessed CPG algorithms that can rank k critical paths efficiently and accurately. However, they all suffer from the lack of *incrementality*, which is the ability to quickly update critical paths after the circuit is incrementally modified. To solve this problem, we introduce Ink, an efficient incremental CPG algorithm. Inspired by the large path trace similarity between adjacent CPG queries, Ink identifies a set of paths to reuse for the next query and effectively prunes the path search space. We have demonstrated the promising performance of Ink on large circuit benchmarks. Ink is up to 22.4× faster and consumes up to 31% less memory than a state-of-the-art timer when generating one million paths on a large design.

ACM Reference Format:
Che Chang, Tsung-Wei Huang, Dian-Lun Lin, Guannan Guo, and Shiju Lin. 2024. Ink: Efficient Incremental k-Critical Path Generation. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3655897

1 INTRODUCTION

Critical Path Generation (CPG) is a key routine in static timing analysis (STA) applications. For example, a practical timer counts on CPG to perform path-based analysis (PBA), such as common path pessimism removal (CPPR) and advanced on-chip variation (AOCV) update, for removing unwanted pessimism [1]. As the design complexity continues to grow, CPG runtime can become a significant bottleneck in many STA engines [5]. To alleviate this problem, academia has introduced various CPG algorithms that can rank k critical paths efficiently. For example, iTimerC introduces a branch-and-bound technique to prune redundant path traversals [6]; iitRace introduces a pin coloring scheme to perform efficient path reduction [7]; OpenTimer introduces a fast implicit path representation algorithm using suffix tree and prefix tree [2].

Although existing CPG algorithms have demonstrated efficiency and accuracy, they all suffer from the lack of *incrementality*, which is the ability to quickly update critical paths after the circuit is incrementally modified. Incrementality plays an important role in many optimization flows, such as timing-driven placement and gate sizing [5]. Figure 1 shows two critical paths before and after a gate sizing operation that incrementally modifies the circuit. Despite different slack values, critical path traces exhibit a large similarity between the two CPG queries (e.g., the second most critical path trace does not change). In fact, according to [5], the overlap ratio of path traces between adjacent incremental timing iterations can go up to 90%. This implies that many path results computed in the previous CPG query are highly reusable for the next CPG query. Without incrementality, CPG algorithms will waste substantial time and memory on recomputing the same paths.

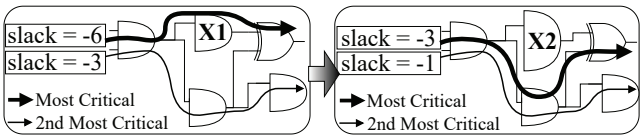


Figure 1: Illustration of CPG (k = 2) for a gate sizing operation (X1→X2). The second most critical path trace is unaffected.

However, designing a fast incremental CPG algorithm is very challenging because we need to efficiently identify which paths to keep and reuse for the next CPG query after the circuit is modified. When those paths are identified, we need to effectively prune them from the search space to avoid duplicated paths. To overcome these challenges, we introduce Ink, an efficient incremental CPG algorithm. Ink is inspired by the implicit path representation algorithm of OpenTimer [2] (suffix and prefix trees), but redesigns its core search routine to efficiently support incrementality. We summarize three technical contributions of Ink as follows:

- We design a fast incremental suffix tree update algorithm that minimally identifies the affected subgraph of the suffix tree and performs only the necessary updates on shortest path values.
- We design a fast incremental prefix tree expansion algorithm that identifies a set of paths to reuse for the next CPG query. With these paths, we can effectively prune the path search space.
- We give rigorous analysis to justify the correctness and complexity of the proposed algorithms.

We evaluate Ink's performance on real circuit benchmarks generated by a state-of-the-art timer, OpenTimer [2]. Compared to OpenTimer's CPG algorithm [2], Ink is up to 22.4× faster and consumes up to 31% less memory when generating one million critical paths on a large design.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3655897

2 BACKGROUND

2.1 Incremental Critical Path Generation

The circuit network is input as a directed-acyclic graph $G = \{V, E\}$. V is a set of n vertices that represent pins of circuit components (e.g., logic gates, flip-flops, etc.). E is a set of m edges that represent pin-to-pin connections. Each edge e is directed from its head vertex u to tail vertex v and is associated with a delay w_e. A path is an ordered sequence of edges $\langle e_1, e_2, ..., e_i \rangle$. The path delay is the summation of delays through all edges of that path. A circuit modifier is an operation that modifies the circuit to perform timing-driven optimization. In this paper, we target the circuit modifier that only alters the edge weights of the graph, which is a specific yet widely used scenario.

Given a circuit graph G and a positive integer k, a CPG query reports the top-k critical paths in ascending order of path slack (or path delay depending on how the graph is formulated [2]). An incremental iteration is defined as at least one circuit modifier followed by one CPG query.

2.2 Implicit Path Representation

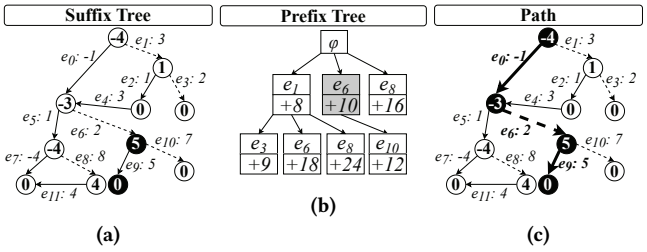


Figure 2: Implicit path representation using suffix tree and prefix tree. Suffix $\langle e_9 \rangle$ + Prefix $\langle e_0, e_6 \rangle$ = Path $\langle e_0, e_6, e_9 \rangle$.

Although there are many CPG algorithms [2, 6, 7], we adopt the implicit path representation algorithm proposed by OpenTimer [2], which outperforms existing algorithms in space and time complexity. As shown in Figure 2, OpenTimer represents critical paths using two complementary data structures, *suffix tree* and *prefix tree*. A suffix tree is a shortest path tree rooted at the destination vertices, constructed with topological relaxations. Figure 2(a) shows an example graph and its suffix tree. Solid edges denote the suffix tree, and dashed edges denote non-suffix tree edges. Numbers on the vertices denote the shortest distance to their destination vertices.

A prefix tree is a tree order of non-suffix tree edges. Each prefix tree node implicitly represents a path deviated from its parent path. The prefix tree root refers to the shortest path in the suffix tree. Figure 2(b) shows an example. The prefix tree root φ implicitly represents the shortest path $\langle e_0, e_5, e_7 \rangle$ in the suffix tree. The prefix tree node marked by "e_6" (colored in gray) implicitly represents the path with prefix $\langle e_0 \rangle$ from its parent path deviated on e_6 and followed by suffix $\langle e_9 \rangle$ from the suffix tree. Figure 2(c) illustrates this path as bold edges $\langle e_0, e_6, e_9 \rangle$. To retrieve the path delay, we record the "deviation cost" of each non-suffix tree edge e: $dvi[e] = dis[tail[e]] + weight[e] - dis[head[e]]$, where $dis[v]$ denotes the shortest distance from vertex v to its destination vertex. Intuitively, deviation cost measures the distance loss by deviating on edge e instead of taking the ordinary shortest path to the destination vertex. For example, in Figure 2(a), e_6 has a deviation cost of $dis[tail[e_6]] + weight[e_6] - dis[head[e_6]] = 10$, which means by deviating on e_6, we get a path that is 10 longer than the shortest path from $head[e_6]$ to its destination vertex. To conclude, Table 1 lists the data fields to which we apply for each prefix tree node [2].

Constructor	PfxtNode(p, e, w)	RespurListItem(pfx, pes)
Members	p: parent node e: deviation edge w: cumulative $dvi[e]$	pfx: prefix tree node pes: pruned edges for pfx

Table 1: Data fields of a prefix tree node (PfxtNode) and a re-spur list item (RespurListItem).

3 INK: INCREMENTAL k-CRITICAL PATH GENERATION

Ink has two stages, *incremental suffix tree update* and *incremental prefix tree expansion*, to perform incremental CPG.

3.1 Incremental Suffix Tree Update

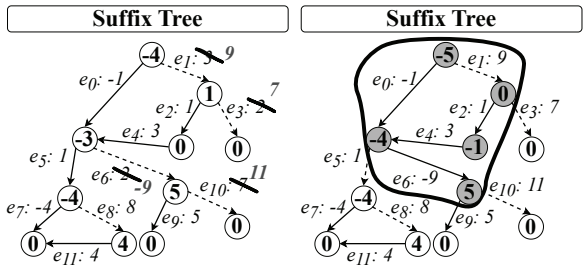
The goal of incremental suffix tree update is to perform only necessary topological relaxations on the affected subgraph of the suffix tree, as opposed to the complete bottom-up topological relaxations in OpenTimer [2]. Algorithm 1 presents the incremental suffix tree update algorithm. After collecting an array of head vertices M from user-modified edges, we perform DFS on M to identify the affected vertices V in reversed topological order (line 2). We record the affected prefix tree nodes for the second stage (line 5:6) and perform edge relaxations on the fanouts of each vertex in V (line 7).

Following the suffix tree example in Figure 2(a), Figure 3(a) shows that we modify the weights of e_1, e_3, e_6, and e_{10}. Figure 3(b) shows that after performing DFS on the head vertices of the modified edges, we identify five affected vertices (marked in gray). We then perform edge relaxations on the fanouts of these five vertices. For example, as shown in Figure 3(b), we perform edge relaxations on e_5 ($dis[tail[e_5]] + weight[e_5] = -3$) and e_6 ($dis[tail[e_6]] + weight[e_6] = -4$). Since $-3 > -4$, -4 becomes $head[e_6]$'s new shortest distance to its destination vertex. $tail[e_6]$ is the new successor of $head[e_6]$. Lemma 1 concludes Algorithm 1.

Lemma 1. *Algorithm 1 takes $O(n + km)$ time complexity.*

Algorithm 1: IncSfxt(M)

```
Input: array of head vertices of user-modified edges M
Global: array of affected prefix tree nodes P
1 P ← ϕ;
2 V ← DFS on M to identify affected vertices in reversed
    topological order;
3 Foreach u ∈ V
4 |   Foreach e ∈ fanout(u)
5 |   |   Foreach n ∈ dependent_pfxt_nodes(e)
6 |   |   |   P ← P ∪ n;
7 |   |   Relax(u, tail[e], weight[e]);
```



(a) Edge weight modifications (b) Updated subgraph

Figure 3: Illustration of Algorithm 1. We only perform topological relaxations on the fanouts of the gray vertices in (b).

3.2 Incremental Prefix Tree Expansion

After updating the suffix tree, the next step is to explore paths that deviate from the suffix tree by expanding the prefix tree. To be clear, "expand" means to generate the children nodes for a certain prefix tree node by finding non-suffix tree edges to deviate on. When a timing-driven application queries k critical paths (potentially very large k), expanding the prefix tree becomes very expensive if not done incrementally. However, incremental prefix tree expansion has two major challenges: 1) we need to know which prefix tree nodes are reusable after applying the circuit modifiers and 2) after identifying these nodes, we need to prune them from the search space for the next query to avoid generating duplicated nodes. To overcome challenge 1, we introduce a theorem that serves as the cornerstone of our incremental prefix tree expansion algorithm:

Theorem 1. *Given a prefix tree node p and p's children C, and each child $c_i \in C$ is associated with an edge e_i, where i represents the order in which c_i is discovered. $\forall i, j \in \mathbb{Z}_{>0}$, if $i < j$ and e_j becomes a suffix tree edge after the circuit is changed, then c_i remains p's child.*

Proof. Assume c_i is not p's child, we examine two cases: 1) if e_i and e_j have the same head vertex v, e_i must be a suffix tree edge, which contradicts the fact that e_j is the only suffix edge among v's fanouts. 2) if e_i and e_j have different head vertices, since c_j is discovered later than c_i, c_i is not affected. Thus, by contradiction Theorem 1 is correct. □

Intuitively, Theorem 1 states that if c_j is associated with a suffix tree edge after the circuit is changed (meaning that c_j will disappear from the prefix tree in the next CPG query), we can reuse c_j's left siblings because they are discovered before c_j and removing c_j does not affect them. We only need to update these siblings' cumulative deviation costs. Since Theorem 1 applies to every level of the prefix tree, we can maximize the number of reusable nodes and reduce memory reallocation overhead. To overcome challenge 2, we maintain a "re-spur list" that records which nodes need re-expansion. For each of these nodes, to avoid generating duplicated children nodes, we also record which edges to skip during re-expansion. Table 1 lists the data field to which we apply for each re-spur list item. *pes* records what edges we should skip when generating the children nodes for *pfx*.

Algorithm 2 describes a key subroutine of Ink, MarkPfxtNodes. The goal of Algorithm 2 is to categorize the prefix tree nodes into reusable and removed nodes by applying Theorem 1. We update

Algorithm 2: MarkPfxtNodes(P, Q)

```
   Input: array of affected prefix tree nodes P, queue Q
   Output: re-spur list R
1  Sort P in ascending order of level;
2  R, pes ← ϕ;
3  Foreach p ∈ P
4      if p is updated or p is removed then
5          continue;
6      Foreach s ∈ siblings(p)
7          Q.push(s);
8      while Q is not empty
9          n ← Q.pop();
10         if n.parent ∈ a re-spur list item then
11             mark n as removed;
12         if n is not removed then
13             if tail[n.e] = successor[head[n.e]] then
14                 mark n as removed;
15                 if n.parent ∉ a re-spur list item then
16                     r ← new RespurListItem(n.parent, pes);
17                     R ← R ∪ r;
18                     clear pes;
19             else
20                 update n.w and mark n as updated;
21                 pes ← pes ∪ n.e;
22         Foreach c ∈ n.children
23             Q.push(c);
24             if n is removed then
25                 mark c as removed;
26 return R;
```

the cumulative deviation costs of the reusable nodes and mark others for lazy removal. Note that Algorithm 2 only prepares Ink for incremental prefix tree expansion by generating the re-spur list; the actual expansion happens in Algorithm 4. To ensure top-down traversal of the affected prefix tree nodes, we sort the array of affected prefix tree nodes P in ascending order of level (line 1). We initialize a re-spur list R and a set of pruned edges *pes* (line 2). For each node in P, if unmarked (line 4:5), we push its siblings to a queue Q to perform BFS (line 6:7). This is because Theorem 1 requires us to visit these nodes in the same order as they are discovered. We pop a node n from Q (line 9). If n's parent is already in the re-spur list (line 10), implying that a left sibling of n is marked as removed, we mark n as removed too (line 11), since n is discovered later than this sibling. If n is unmarked (line 12), we check if $n.e$ is a suffix tree edge (line 13). If so, n disappears from the prefix tree, and we mark n as removed (line 14). We create a re-spur list item (line 16:17), indicating that n's parent will later expand but skip *pes*. Otherwise, we update n's cumulative deviation cost and add n's edge to its parent's *pes* (line 20:21). We finally enqueue n's children for the later BFS iterations (line 22:25).

Continuing from the updated suffix tree in Figure 3(b), Figure 4 illustrates Algorithm 2. We denote a prefix tree node associated with e_i and cumulative deviation cost w as PfxtNode(e_i, w). For simplicity, we leave out the *parent node* member mentioned in

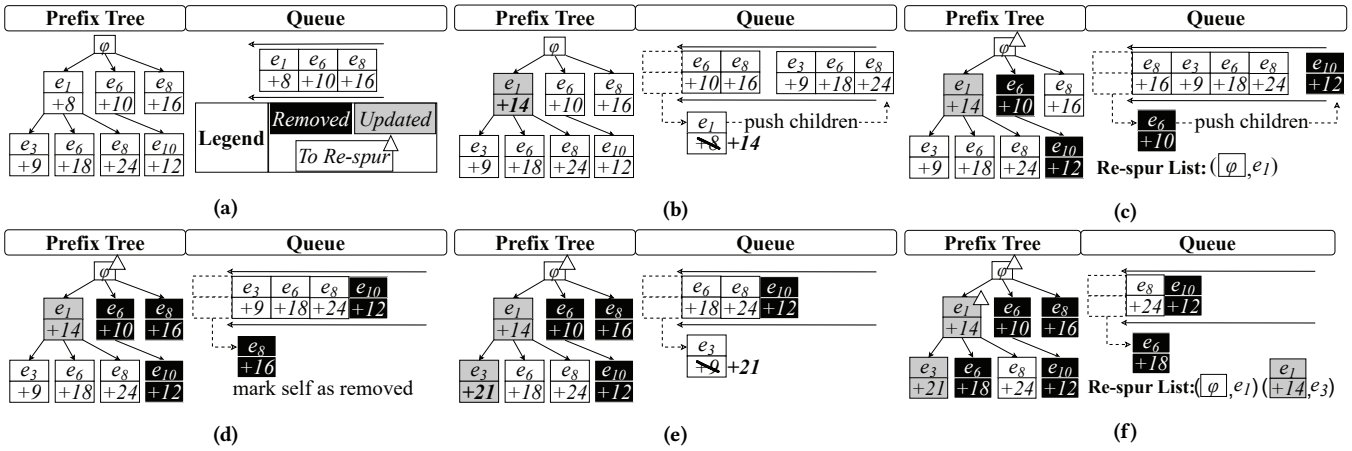


Figure 4: Illustration of Algorithm 2 (continuation of Figure 3(b)), a key subroutine of the proposed incremental prefix tree expansion algorithm. (a) Prefix tree and a queue that has PfxtNode(e_1, 8) and its siblings. (b) e_1 is still a non-suffix tree edge, so we update PfxtNode(e_1, 8)'s cumulative deviation cost to 14. (c) e_6 is now a suffix tree edge, so we mark PfxtNode(e_6, 10) and its children as removed. We then create a re-spur list item indicating that φ will skip e_1 during re-expansion. (d) e_8 is discovered later than e_6, so we mark PfxtNode(e_8, 16) and its children as removed. (e) Similar to (b), we update PfxtNode(e_3, 9)'s cumulative deviation cost to 21. (f) Similar to (c), we mark PfxtNode(e_6, 18) as removed. We then create a re-spur list item indicating that PfxtNode(e_1, 14) will skip e_3 during re-expansion.

Table 1, since it is already illustrated. Figure 4(a) illustrates the prefix tree for four paths and a queue containing PfxtNode(e_1, 8) and its siblings. Note that a four-critical path query may generate more than four nodes [2], so we see eight nodes in Figure 4(a). Figure 4(b) illustrates that we pop PfxtNode(e_1, 8) from the queue. Since e_1 is still a non-suffix tree edge, PfxtNode(e_1, 8) remains φ's child. We update PfxtNode(e_1, 8)'s cumulative deviation cost to $0+9-(-5) = 14$ using the shortest path values in Figure 3(b). We also push PfxtNode(e_1, 14)'s children to the queue. Figure 4(c) illustrates that we pop PfxtNode(e_6, 10) from the queue. Since e_6 is now a suffix tree edge, PfxtNode(e_6, 10) should be removed. We create a re-spur list item indicating that PfxtNode(e_6, 10)'s parent φ will skip e_1 during re-expansion. We should remove PfxtNode(e_6, 10)'s children as well, and we push them to the queue. Figure 4(d) illustrates that we pop PfxtNode(e_8, 16) from the queue. PfxtNode(e_8, 16)'s parent φ belongs to a re-spur list item, indicating that one of PfxtNode(e_8, 16)'s left siblings is removed. Since PfxtNode(e_8, 16) is discovered later than this removed sibling, we remove PfxtNode(e_8, 16) and its children. Figure 4(e)–(f) repeat the same procedure and finally produce two re-spur list items. Lemma 2 concludes Algorithm 2.

Lemma 2. *Algorithm 2 takes $O(k \log k)$ time complexity.*

Algorithm 3 describes another subroutine, which redesigns the Spur algorithm in [2] to support incrementality. Algorithm 3 expands the prefix tree from a given prefix tree node. Our algorithm includes a set of pruned edges *pes* as input, which allows us to minimally expand the prefix tree from a given node by pruning *pes* during expansion (lines 1 and 5). Lemma 3 concludes Algorithm 3.

Lemma 3. *Algorithm 3 takes $O(n + m \log k + k)$ time complexity.*

Using Algorithms 2–3 as primitives, Algorithm 4 describes the incremental prefix tree expansion algorithm. The goal of Algorithm 4 is to retrieve the top-k critical paths in ascending order of path delay by incrementally expanding the prefix tree. Since we are

Algorithm 3: SpurPruned(*pfx*, *d*, $\hat{Q}$, *pes*)

Input: a prefix tree node *pfx*, destination vertex *d*, priority queue $\hat{Q}$, a set of pruned edges *pes*

```
1  mark all edges in pes as pruned in the given graph;
2  u ← tail[pfx.e];
3  while u ≠ d
4      Foreach e ∈ fanout(u)
5          if tail[e] = successor[u] or e is pruned then
6              continue;
7          pfx_new ← new PfxtNode(pfx, e, pfx.w + dvi[e]);
8          Q̂.enqueue(pfx_new);
9      u ← successor[u];
10 unmark all edges in pes in the given graph;
```

retrieving paths incrementally, we transfer the essential information from the previous CPG query, including a priority queue $\hat{Q}$ of nodes keyed on their cumulative deviation costs (line 1) and the dequeued nodes Λ (line 2). We initialize the solution path set and a queue Q (line 3). We generate a re-spur list R using Algorithm 2 (line 4). Since Algorithm 2 invalidates $\hat{Q}$'s heap property, we heapify $\hat{Q}$ (line 5). With R, we can reuse updated nodes from the previous CPG and minimally expand the prefix tree (line 6:7). In OpenTimer [2], this critical path retrieval procedure always satisfies the condition where the nodes in Λ have cumulative deviation costs no more than the minimum cumulative deviation cost in $\hat{Q}$. However, Algorithm 2 may cause Λ to violate this condition. To solve this, we recover unremoved paths from Λ and record the maximum cumulative deviations cost *max_dc* in Λ (line 8); we also expand any leaf nodes in Λ, because they may have undiscovered children. If in the path search loop (line 9:18), we see a node that has a cumulative deviation

cost less than *max_dc* (line 16), meaning the above condition is still violated, we continue executing the loop. The path search loop iteratively dequeues a node *pfx* (line 10), recovers the path (line 14:15), and then expands the search space for *pfx* (line 18) until we retrieved enough paths and the above condition is fulfilled. Combining Lemma 2–3, we draw the following theorem.

Theorem 2. *Algorithm 4 takes $O(n+m+k)$ space complexity and $O(kn+km\log k+k^2)$ time complexity.*

Proof. The space complexity of Algorithm 4 involves $O(n+m)$ for storing the circuit graph, $O(n)$ for the suffix tree, $O(k)$ for the prefix tree, and $O(k)$ for the re-spur list. Hence, the total space complexity is $O(n+m+k)$. We perform Algorithm 3 up to k iterations to obtain the top-k critical paths. Therefore, the total time complexity is $O(kn+km\log k+k^2)$. □

Algorithm 4: IncPfxt(d, k, P)

Input: destination vertex d, path count k, affected prefix tree nodes P
Output: solution set Ψ of critical paths

1 $\hat{Q} \leftarrow$ priority queue of nodes from the previous CPG;
2 $\Lambda \leftarrow$ transfer dequeued nodes from the previous CPG;
3 $\Psi, Q \leftarrow \phi$
4 $R \leftarrow$ MarkPfxtNodes(P, Q);
5 $\hat{Q}$.heapify();
6 **Foreach** $r \in R$
7 SpurPruned($r.pfx$, d, $\hat{Q}$, $r.pes$);
8 *num_paths*, *max_dc*, $\Psi \leftarrow$ recover paths from nodes that are unremoved in Λ and record max cumulative deviation cost;
9 **while** $\hat{Q}$ is not empty
10 $pfx \leftarrow \hat{Q}$.dequeue();
11 **if** *pfx* is removed **then**
12 **continue**;
13 $num\_paths \leftarrow num\_paths + 1$;
14 $path \leftarrow$ recover path from pfx;
15 $\Psi \leftarrow \Psi \cup path$;
16 **if** $pfx.w \geq max\_dc$ **and** $num\_paths \geq k$ **then**
17 **break**;
18 SpurPruned(pfx, d, $\hat{Q}$, ϕ);
19 **return** Ψ;

4 EXPERIMENTAL RESULTS

We implemented Ink in C++ and compiled it with GCC 11.4.0 on a 4.8-GHz 64-bit Linux machine of an Intel Core i5-13500 Processor. We enable the optimization flag `-O3` and C++17 standard `-std=c++17`. We select seven large circuits generated by OpenTimer [2] to evaluate Ink's performance. We only compare the proposed algorithms with OpenTimer beacuse its CPG algorithm outperformed existing methods.

4.1 Overall Performance Comparison

Table 2 compares the suffix tree update runtime, prefix tree expansion runtime, total runtime, and memory usage between full CPG and incremental CPG (Ink) on seven circuits. For each circuit, we measure the performance of Ink by taking the average of 100 incremental iterations that simulate a gate-sizing optimization algorithm developed atop OpenTimer [2]. For wb_dma, tv80, ac97_ctrl, aes_core, and des_perf, we use their maximum path counts for each CPG call. For vga_lcd and netcard, whose maximum path counts are enormous, we use sufficiently large path counts (one million and five million) for each CPG call. Each incremental iteration randomly resizes a gate to alter the edge weight of the circuit graph and issue a CPG call to trigger a timing update. Full CPG refers to the update that re-runs the whole CPG without incrementality, which is how OpenTimer [2] deals with circuit graph updates, while incremental CPG refers to the proposed method. As shown in Table 2, Ink outperforms full CPG in all circuits. Since Ink partially reuses the previous CPG results, it is faster and uses less memory than full CPG. For example, Ink is 22.4× faster and uses 31% less memory in vga_lcd. We do not compare accuracy because our algorithms can produce the same solutions as the golden solutions produced by OpenTimer.

Figure 5 plots the runtime distribution of full CPG and Ink across 50 incremental iterations. Depending on the circuit modifier, the runtime per incremental iteration can vary. Regardless of the variation, we see a consistent runtime gap between full CPG and Ink. Taking netcard as an example, Ink is 8.3× faster than full CPG at the 22^{nd} incremental iteration.

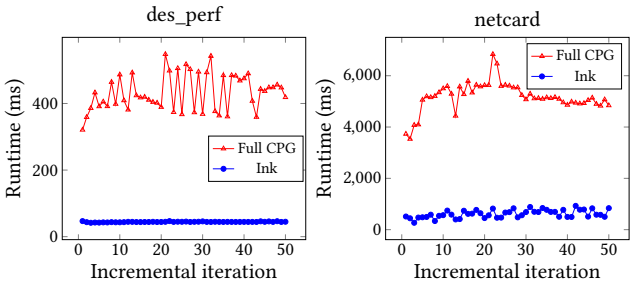


Figure 5: Runtime distribution of full CPG and Ink across 50 incremental iterations for des_perf and netcard.

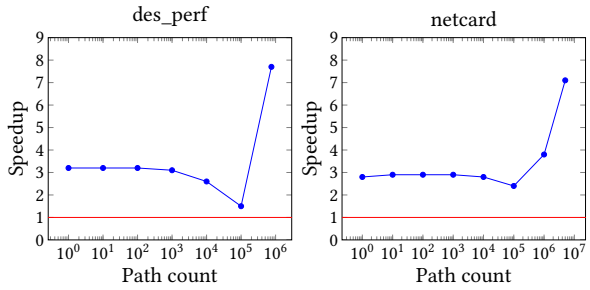


Figure 6: Speedup vs path count for des_perf and netcard.

4.2 Performance at Different Path Counts

Figure 6 demonstrates the speedup of Ink over full CPG at different path counts for des_perf and netcard. As we increase the path count,

Table 2: Overall performance comparison between full CPG (OpenTimer [2]) and incremental CPG (Ink).

Circuit	$\|V\|$	$\|E\|$	Path count (K)	Full CPG (OpenTimer [2]) Sfxt (ms)	Pfxt (ms)	Total (ms)	Mem (MB)	Incremental CPG (Ink) Sfxt (ms)	Pfxt (ms)	Total (ms)	Mem (MB)
wb_dma	12602	8184	32	1.3	3.9	5.2	23.1	0.4 (3.3×)	0.6 (6.5×)	1 (**5.2**×)	17.8 (-23%)
tv80	16681	11364	45	2	6.3	8.3	30.4	0.5 (4×)	1.1 (5.7×)	1.6 (**5.2**×)	22.6 (-26%)
ac97_ctrl	40210	25803	103	7	19.4	26.4	64.3	1.7 (4.1×)	3 (6.5×)	4.7 (**5.6**×)	47.2 (-27%)
aes_core	66221	43022	172	13.2	56.1	69.3	104.7	3.3 (4×)	6 (9.4×)	9.3 (**7.5**×)	75.9 (-28%)
des_perf	295808	189276	757	82.1	260.3	342.4	447.1	13.4 (6.1×)	30.8 (8.5×)	44.2 (**7.7**×)	320.8 (-28%)
vga_lcd	397806	473772	1000	99.6	712	811.6	778.7	5.7 (17.5×)	30.5 (23.3×)	36.2 (**22.4**×)	538.5 (-31%)
netcard	3901343	2402788	5000	1612.4	3012.1	4624.5	4308.9	440.2 (3.7×)	209.5 (14.4×)	649.7 (**7.1**×)	3466.2 (-20%)

Sfxt: suffix tree update runtime Pfxt: prefix tree expansion runtime

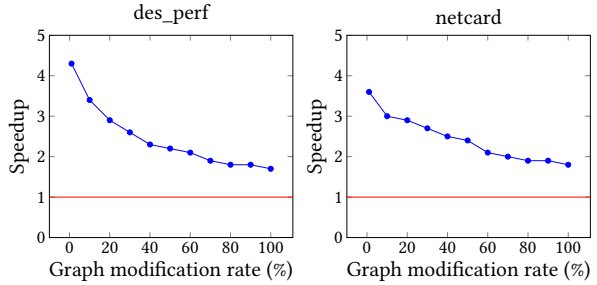


Figure 7: Speedup vs incrementality for des_perf and netcard.

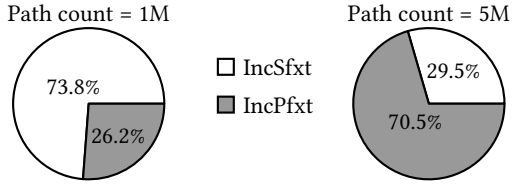


Figure 8: Speedup breakdown of Algorithm 1 (IncSfxt) and Algorithm 4 (IncPfxt) at different path counts.

the speedup of Ink first decreases and then increases after a certain path count. For example, in des_perf, the speedup decreases from over 3× to less than 2× between one path and 100K paths, and then the speedup increases after 100K paths. This is because when the path count is small, Algorithm 1 is the major contributor to Ink's overall speedup. As we increase the path count, prefix tree expansion starts to dominate the performance, but the path count is not large enough for Algorithm 4 to become effective; thus, Ink's overall speedup decreases. As we further increase the path count, Algorithm 4 exhibits a large speedup over full prefix tree expansion; thus, Ink's overall speedup increases.

4.3 Performance at Different Incrementalities

Figure 7 demonstrates the speedup of Ink over full CPG at different graph modification rates for des_perf and netcard. As we increase the graph modification rate, the speedup drops accordingly. For example, Ink's speedup drops from 3.6× to 1.8× in netcard. This is because the higher the graph modification rate, the more nodes that Ink needs to visit in Algorithm 2. Ink is most effective at a low graph modification rate. For example, Ink is over 4× faster in des_perf at 1% graph modification rate. This emphasizes Ink's benefit because realistically one incremental iteration involves only modifying far less than 1% of the gates in the circuit. On the contrary, Ink is still faster at 100% graph modification rate. For example, Ink is almost 2× faster in netcard at 100% graph modification rate. This is because even if the whole circuit is updated, it is very likely that many critical path traces remain the same. Ink only needs to update the path delays, which largely reduces memory reallocation overhead.

4.4 Speedup Breakdown of IncSfxt and IncPfxt

Figure 8 demonstrates the speedup breakdown of Algorithm 1 (IncSfxt) and Algorithm 4 (IncPfxt) for netcard. As we increase the path count from one million to five million, the speedup of Algorithm 4 becomes more remarkable. For example, the speedup of Algorithm 4 increases from 26.2% to 70.5%. This is because the efficiency of Algorithm 1 is constrained by the size of the affected subgraph of the suffix tree.

5 CONCLUSION

In this paper, we have introduced Ink, an efficient incremental k-critical path generation algorithm. Compared to a state-of-the-art timer, Ink is up to 22.4× faster and consumes up to 31% less memory when generating one million critical paths on a large design. We plan to extend Ink to a parallel target using [3, 4].

ACKNOWLEDGMENTS

This project is supported by NSF grants 2235276, 2349144, 2349143, 2349582, and 2349141.

REFERENCES

[1] Jayaram Bhasker and Rakesh Chadha. 2009. *Static Timing Analysis for Nanometer Designs: A Practical Approach.* Springer.
[2] Tsung-Wei Huang, Guannan Guo, Chun-Xun Lin, and Martin D. F. Wong. 2021. OpenTimer v2: A New Parallel Incremental Timing Analysis Engine. *IEEE TCAD* (2021).
[3] Tsung-Wei Huang, Chun-Xun Lin, Guannan Guo, and Martin Wong. 2019. Cpp-Taskflow: Fast Task-based Parallel Programming using Modern C++. In *IEEE IPDPS*. 974–983.
[4] Tsung-Wei Huang, Dian-Lun Lin, Chun-Xun Lin, and Yibo Lin. 2022. Taskflow: A Lightweight Parallel and Heterogeneous Task Graph Computing System. In *IEEE TPDS*, Vol. 33. IEEE, 1303–1320.
[5] Tsung-Wei Huang and Martin D. F. Wong. 2015. OpenTimer: A High-Performance Timing Analysis Tool. In *IEEE/ACM ICCAD*. 895–902.
[6] Pei-Yu Lee, Iris Hui-Ru Jiang, Cheng-Ruei Li, Wei-Lun Chiu, and Yu-Ming Yang. 2015. iTimerC 2.0: Fast incremental timing and CPPR analysis. In *ACM/IEEE ICCAD*.
[7] Chaitanya Peddawad, Aman Goel, Dheeraj B, and Nitin Chandrachoodan. 2015. iitRACE: A memory efficient engine for fast incremental timing analysis and clock pessimism removal. In *ACM/IEEE ICCAD*.